LLMs explained (Part 6): Smarter AI through Reinforcement Learning

Why fine-tuning is not enough and how reinforcement learning with human feedback shapes smarter models.

JOSE PARREÑO GARCIA
JUN 21, 2025

7 7 Share

This week, we are wrapping up our deep dive into LLMs with one of the most fascinating and impactful steps: **reinforcement learning with human feedback (RLHF).**

Throughout this series, we have explored every major phase of building an LLM:

- **Collecting and cleaning data** to ensure models learn from high-quality sources
- **Training a baseline model** and understanding why tokenisation matters
- **Fine-tuning** to make models more useful beyond simple autocomplete
- **Reducing hallucinations** by giving LLMs access to external tools

Now, in this final part, we go beyond training. Fine-tuning gets models far, but it has limits—it cannot teach a model to generalise preferences, adapt to complex human intent, or make better long-term decisions. That is where RLHF comes in, using human feedback to refine how LLMs respond, making them more aligned, reliable, and actually helpful.

Let's dive into how RLHF works, where it succeeds, and the challenges that remain! 🚀

⚡ *Reminder: This series is inspired by Andrej Karpathy's deep dive on LLMs. If you haven't watched it yet, you definitely should!*

Blog series

- ✅ Part 1. LLMs explained: The 3-layer framework behind chatGPT & friends
- ✅ Part 2. LLMs explained: How LLMs collect and clean training data
- ✅ Part 3. LLMs explained: From tokens to training – how a baseli
- ✅ Part 4. LLMs explained: Making LLMs actually useful through fine-tuning

- ✅ Part 5. LLMs explained: Reducing hallucinations by using tools
- 📌 Part 6. LLMs explained: Smarter AI through Reinforcement Learning ***(this post!)***

The limits of fine-tuning

In part 4, we saw how supervised fine-tuning (SFT) is great for task-specific adaptation such as translation, summarisation, and autocomplete. In Part 5, we then explored how fine-tuned LLMs still hallucinate and how adding external tools (like web search and code execution) can help mitigate those issues. Still, SFT has limitations we want to overcome:

It lacks adaptive learning. SFT models can't adjust dynamically to new prompts or values; they are bound by their training data.

- *For example, I would love my ChatGPT to adapt to my style of writing or my answer preferences over time. This can't happen using SFT only.*

It's a one-shot learning process. Once fine-tuned, the model is frozen in production and doesn't improve based on new interactions.

- *For example, what if LLMs could refine their own knowledge from web search? Right now, they can't; FT keeps them locked in their original dataset.*

It optimises for short-term token prediction. Instead of focusing on long-term coherence, models trained solely on SFT aim to predict the next token based on examples from a static dataset.

- *For example, ask an SFT-trained model to write a short story, and it might craft a great opening line. Then, suddenly, it changes character names or forgets the plot. That's because it's optimising for next-word accuracy, not long-term consistency*

An analogy from high-school maths

Imagine a student who has only learned **one** way to solve a math problem. If a test question is slightly different, they struggle—not because they lack intelligence, but because they were only taught **one rigid approach**. Well, this is how SFT-trained LLMs operate. they mimic patterns but can't generalise beyond them.

We need something else

If we want LLMs to improve dynamically, make better long-term decisions, and learn from their own outputs, we need something beyond static fine-tuning.

That is where reinforcement learning comes in: helping models adapt, optimise, and improve even after they have been deployed. And this is what this blog will cover.

What this blog will cover

1. **Reinforcement learning for objective tasks.** *Learning new ways to get to the same answer.*
2. **Reinforcement learning for open-ended answers.** *Where humans enter the reinforcement learning loop.*
3. **Challenges and trade-offs in reinforcement learning.** *We still depend on how humans setup the environment.*
4. **When does reinforcement learning beat supervised fine tuning.** *And where is doesn't.*

Let's dive in!

Reinforcement learning for objective tasks.

Objective tasks, the ones with right/wrong answers (such as science problems, classification or retrieval of specific entities from a text), can be solved purely through SFT. But, as we have just seen, SFT is limited by the input data it is trained on.

By definition, if SFT are carefully curated datasets and examples, then the number of examples that humans can generate has upper bounds. You can't simply create 1 million different ways to solve a specific maths problem by hand. Even less, doing it for the hundred of thousands of problem we want to present the SFT.

But, what if we ask an LLM to generate those 1 million different ways to solve the maths problem, and, also check where it go it right or wrong?

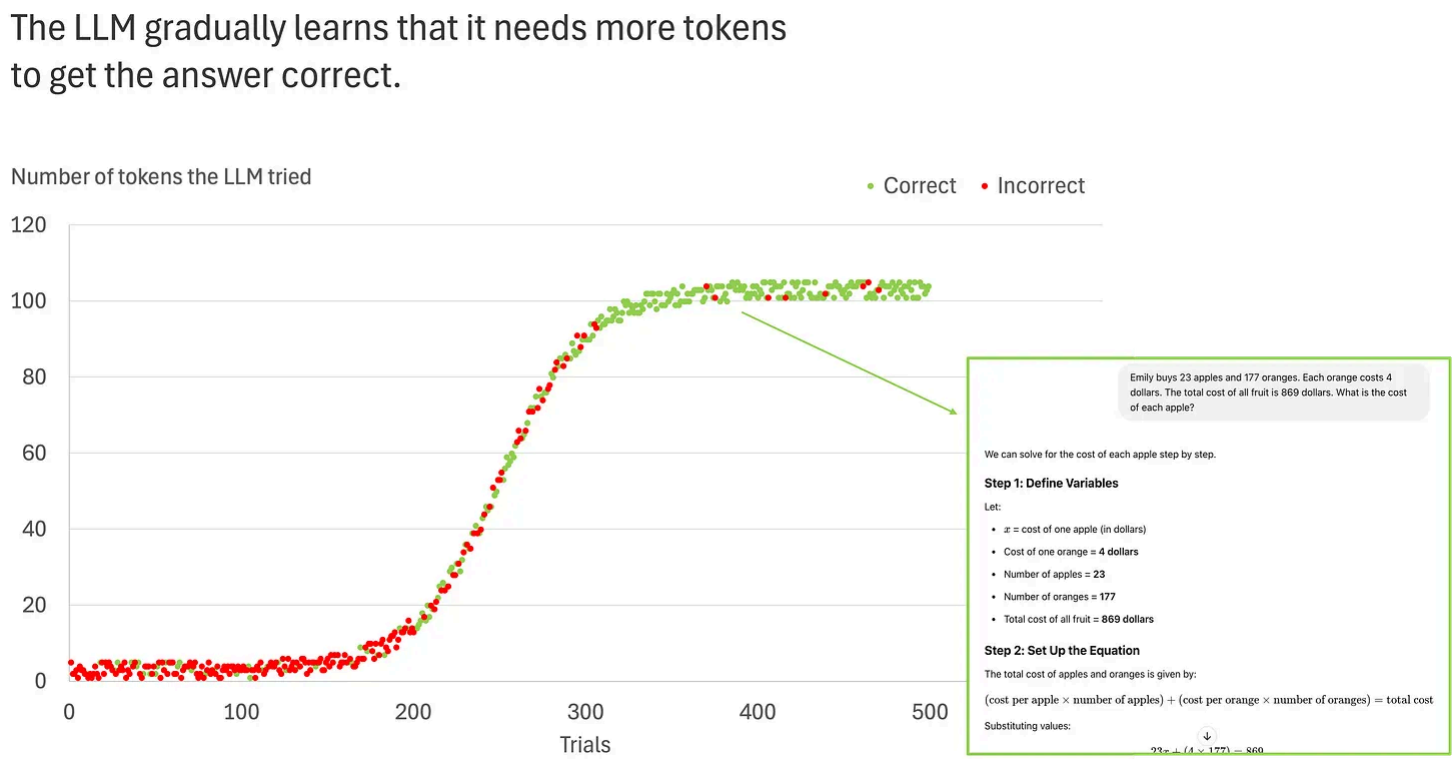
Let's understand how this might help the LLM.

How reinforcement learning improves math problem solving

I won't go into the maths of reinforcement learning in this post. But, I would like you to understand that reinforcement learning is a game of trial and error. Applied to LLMs you can think of it this way:

1. **The LLM has an exploration space.** For example, the LLM can explore to provide answers with 3 tokens (or with 300).
2. **The LLM has a reward.** In this case, after a response is given, the LLM can check if it got the answer right.
3. **The LLM has a reaction to the reward.** Based on formulas, the LLM can start exploring more or less aggressively towards the space (in this case, number of tokens), that tends to provide the highest rewards.

The diagram below illustrates this concept visually. In reality, the LLM's learning curve wouldn't be this perfectly smooth, but this example makes it easier to see how RL works in practice.



What we see is that:

1. The LLM starts exploring the space with few tokens. Maybe it was trained to not use a lot of resources (tokens), so it goes for low numbers.
2. The LLM mostly gets incorrect answers in this token space. This is why it starts to increase the exploration space around the 200th trial.
3. The LLM then starts seeing many more rewards. Therefore, it encourages itself to keep increasing tokens.

4. The LLM finally converges to a token space of ~100 tokens.

What is the model learning?

Over time, RL teaches the LLM that longer reasoning sequences lead to better answers. Not necessarily, that it needs 100 tokens. It's more that, for *this* sort of maths problem, it will understand that statistically speaking, it will not get correct answers with very few tokens.

Is this only applicable to maths?

No, no, no! This is applicable to any problem space that can be factually checked. Chemistry, physics, biology, code generation, logic, entity extraction from text, factual history questions, etc. The problem spaces can vary, but the concept of reinforcement learning and how it helps decide how to answer a question, will still be the same.

Now, this was the "easy" side of reinforcement learning. What about the cases where the answers are not right / wrong? What if we want to improve the LLM writing poems, summarising articles or telling us a joke?

Let's cover how reinforcement learning can be adapted to tackle this scenarios in the following section.

Reinforcement learning for open-ended answers.

So far, we have looked at objective tasks—problems with clear right or wrong answers, where reinforcement learning helps models discover better fact-based solutions. But what about tasks where there's no single correct answer?

The answer: **Reinforcement Learning with *Human Feedback*** (RLHF).

RLHF needs verifiers, not labels

Supervised Fine-Tuning (SFT) relies on labeled data, where human annotators provide explicitly correct answers for the model to learn from. But for open-ended tasks, this doesn't work. For open-ended tasks, **correctness is subjective**.

For example, if an LLM generates a summary of an article, there isn't a single "correct" summary—some might focus on key facts, others on writing style, and some might even introduce biases. Clearly, we need a different way to evaluate LLM outputs.

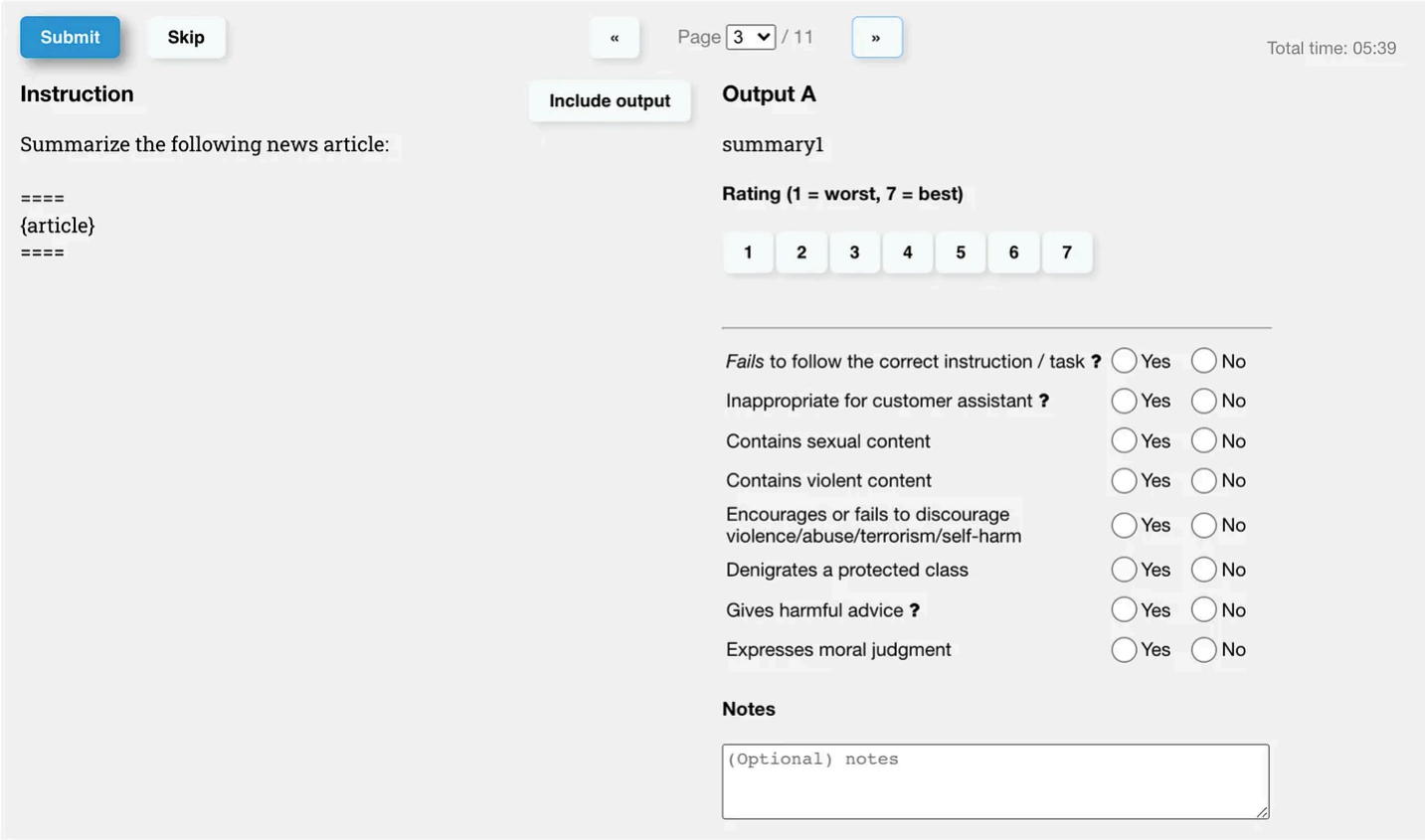
Instead of using fixed labels, Reinforcement Learning with Human Feedback (RLHF) relies on verifiers: ways to score responses based on quality, coherence, or human preference.

How RLHF works for open-ended tasks

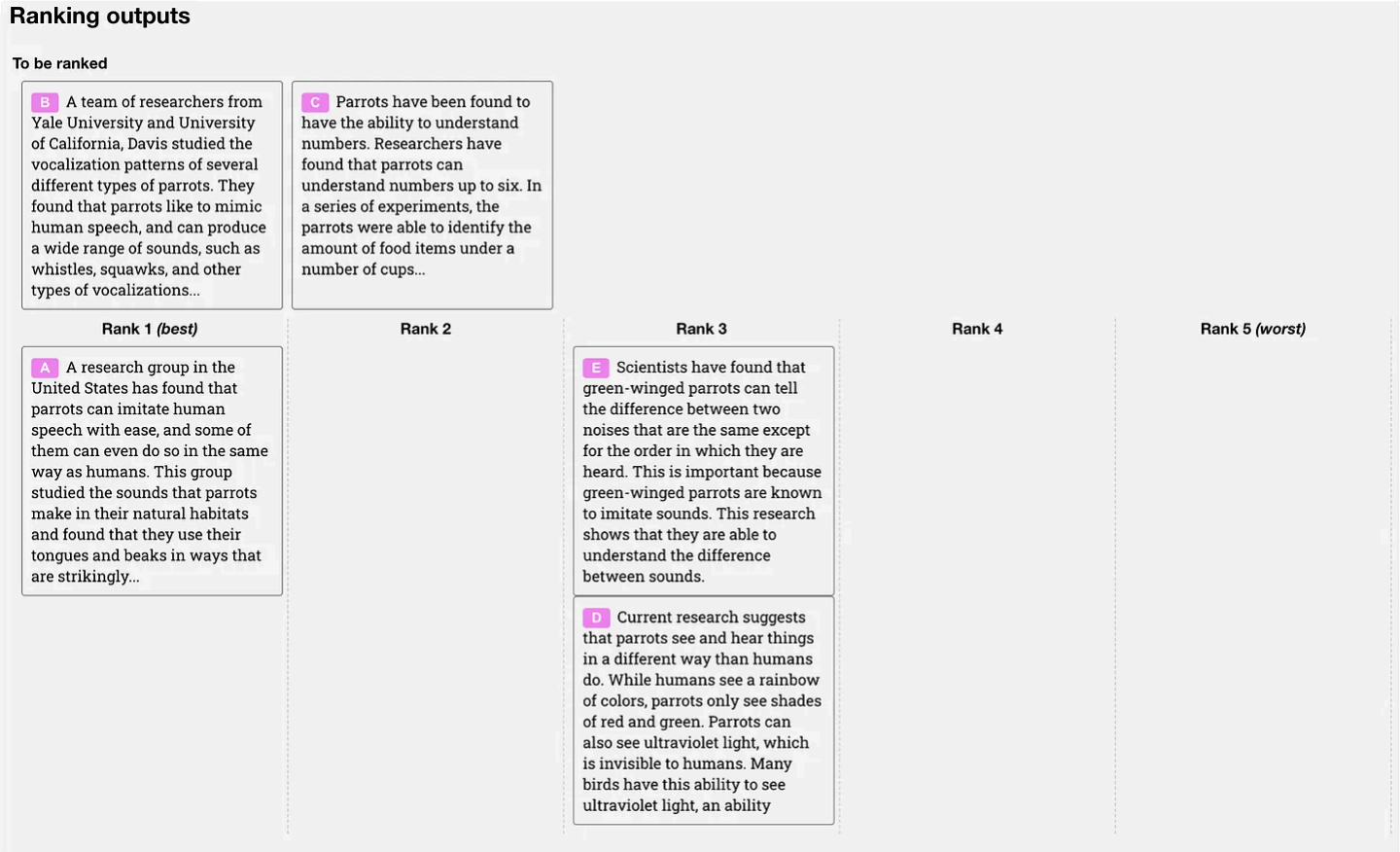
- **Step 1: Generate multiple responses.** The LLM is asked to summarise an article or answer an open-ended question. Instead of producing just 1 response, it generates several different versions.
- **Step 2: Human labellers *rank* the responses.** Instead of marking responses **correct or incorrect**, human annotators **rank them from best to worst** based on many dimensions. For example, relevance to the original prompt, clarity and conciseness, factual accuracy (if applicable), coherence and readability, etc.
- **Step 3: Training a reward model.** The Reward Model (RM) learns from these human rankings and starts predicting which responses humans would *prefer*. Over time, the LLM optimises its responses to align with human preferences, even for subjective tasks.

A real-world example from OpenAI

In its 2022 Training LLMs to follow instructions with human feedback paper, OpenAI shows a fantastic example of how they built a system for human labellers could rank LLM responses and all the data could be collected.



For a summarisation task, the labeller has to rank the output and score it based on 8 dimensions.



The labeller can also review how the responses ranked against it each as ties as encouraged.

How does RLHF improve the model?

Step 1: Human labellers rank LLM responses (what we saw above)

- A labeller reviews multiple model-generated summaries and scores them across eight dimensions (e.g., clarity, factuality, coherence).
- The labeller ranks the responses from best to worst, rather than marking any as strictly “correct” or “incorrect.”

Step 2: Training a reward model

- The Reward Model is a separate neural network trained to predict human rankings.
- Given a set of seven responses, the reward model learns which features make an answer more likely to be ranked highly.

Step 3: Fine-tuning the LLM via reinforcement learning

- Once the reward model can accurately predict rankings, it is used to fine-tune the LLM using Proximal Policy Optimization (PPO).
- The model adjusts itself to maximise the reward model’s predicted score, improving responses dynamically over time

A diagram to visually see all the steps together

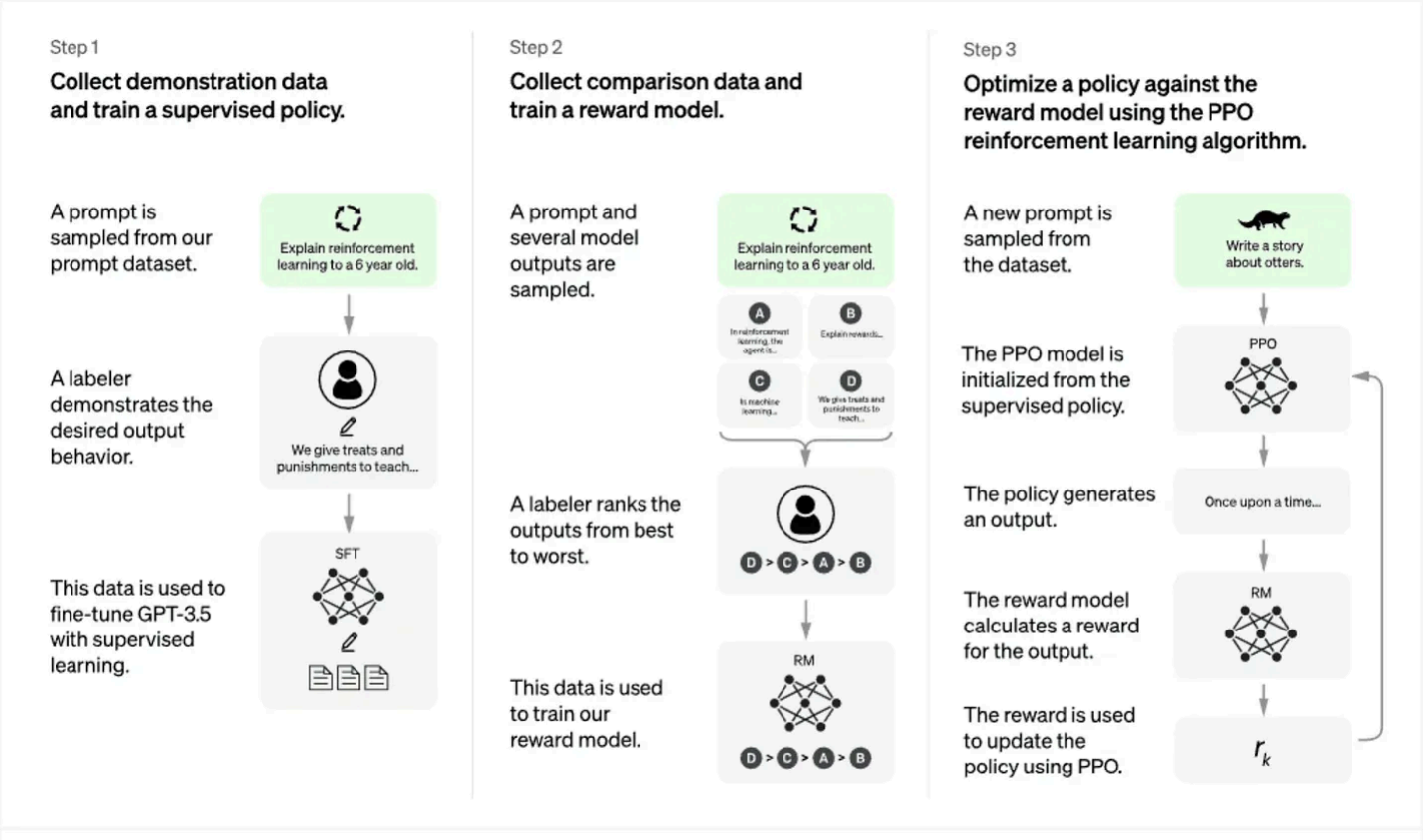


Image source: https://openai.com/index/chatgpt/

Now we now *how* reinforcement learning can improve over supervised fine-tuning. RLHF helps LLMs go from simple correctness to teaching them to align with human preferences even in subjective tasks. In the next section, let's briefly cover some cases where reinforcement learning is a must have!

When does reinforcement learning beat supervised fine tuning.

In certain cases, RFT (Reinforcement Fine-Tuning) and RLHF become essential to achieve better performance.

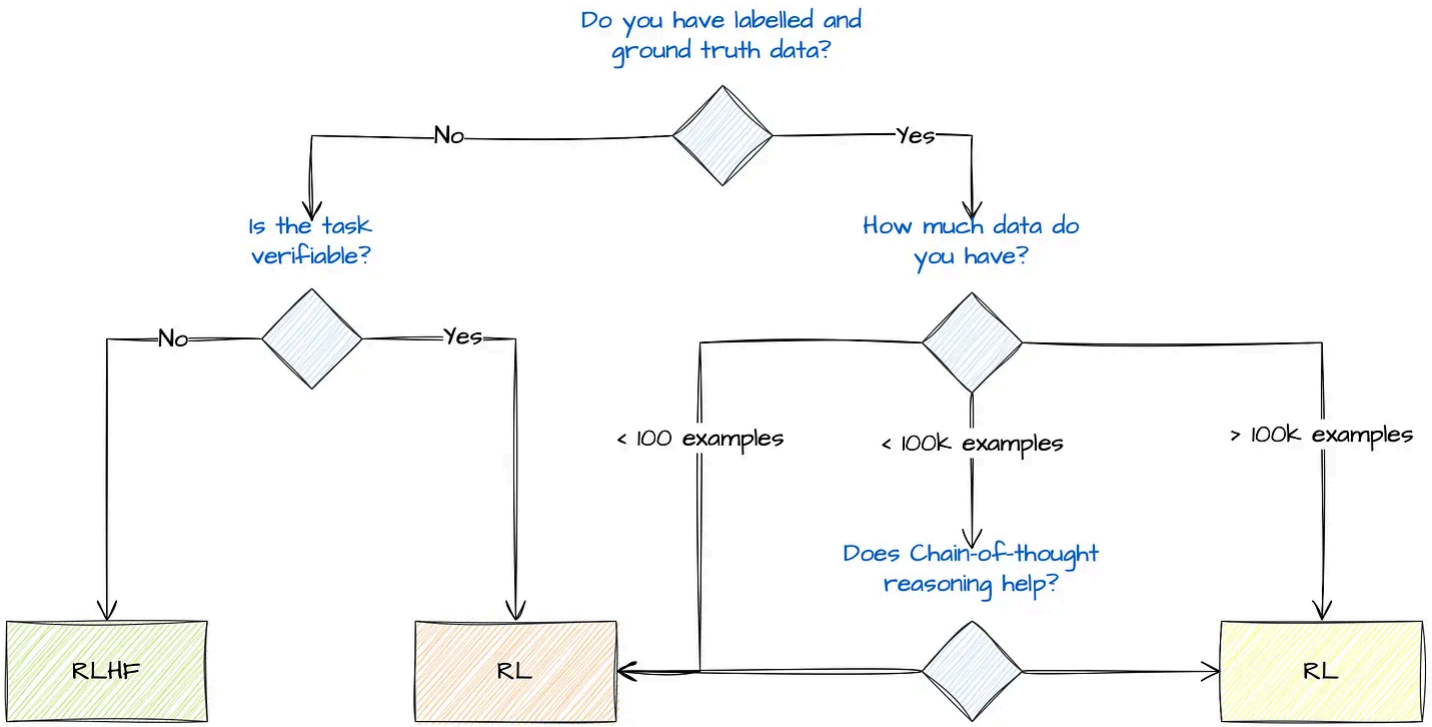
Below are 4 key scenarios where reinforcement learning outperforms supervised fine-tuning, making models more adaptive, more aligned with human intent, and more capable of reasoning.

Scenario	Without RL	With RLHF / RFT
RFT Beats SFT When Data Is Scarce	SFT overfits when trained on just a few dozen examples, memorizing data instead of generalizing.	RFT can learn general patterns from limited examples, making it more efficient when labeled data is scarce. However, SFT is better when large datasets are available (>100k examples).
Aligning Models with Human Preferences	Chatbot responses can feel robotic, off-topic, or unengaging.	RLHF helps models adapt to human tone, style, and intent, making interactions more natural (e.g., formal vs. casual responses).
Improving Code Generation	Model-generated code is syntactically correct but often inefficient or unreadable.	RLHF optimizes for readability, efficiency, and best practices, producing more useful code.
Enhancing Reasoning with Chain-of-Thought	SFT can distill strong reasoning from a larger model into a smaller one, but it cannot surpass the teacher model's baseline reasoning abilities.	RFT enables models to improve their reasoning strategies, making them better at complex multi-step logic (e.g., DeepSeek-R1).

Table: Jose Parreño • Created with Datawrapper

A diagram to understand how SFT, RFT and RLHF fits together

This diagram visually compares how reinforcement learning builds upon fine-tuning for more adaptive AI.



Key takeaways from part 6

1. **Fine-tuning has limits, RLHF fills the gaps.** Supervised fine-tuning (SFT) makes LLMs better at following instructions, but it cannot teach models to generalise

beyond their dataset or adapt dynamically to new tasks.

2. **Reinforcement Learning with Human Feedback (RLHF) makes models more aligned.** Instead of rigidly following fixed labels, RLHF optimises responses based on human preferences, making models more useful for subjective tasks like summarisation and creative writing.
3. **LLMs can learn from rewards, not just examples.** Unlike SFT, which relies on predefined answers, RLHF allows models to experiment, receive feedback, and refine their responses over time—like a student improving through practice and critique.

Series wrap-up! 🎉

And that is a wrap!

If you have made it this far, congratulations—you now have a solid understanding of how LLMs go from raw data to fully functional AI assistants.

From data collection and tokenization to fine-tuning, hallucination reduction, and reinforcement learning, we have covered the essential steps that shape modern LLMs like ChatGPT, Claude, and Gemini. My goal with this series was to demystify the process, showing that while these models may seem like magic, they are built on layers of systematic training, optimisation, and a lot of trial and error.

A huge **THANK YOU** for following along on this journey! Whether you are a data scientist, an engineer, or just someone curious about how AI works under the hood, I hope you have gained valuable insights into the inner workings of LLMs.

If you have enjoyed this series, do not hesitate to **share it with others**, drop a comment, or let me know which topics you would love to see explored next. The world of AI is moving fast, and there is always more to learn.

Until next time—keep exploring, keep questioning, and stay curious. 🚀

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

7 Likes • 7 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture